

Web and Mobile Algorithms

Accurate implementation guide for the Payroll, Attendance, POS, Inventory, and Quotation System

Scope of this document: This is not a Chapter 4 template. It documents only the algorithms currently used by the web and mobile applications, how each algorithm is executed, and the source files where the behavior is implemented.

1. Shared System Flow

```
WEB:    Browser → index.php / ajax.php → Action class → MySQL → HTML/JSON
MOBILE: Screen → Zustand/local database → Axios → ajax.php → Action class → MySQL → JSON → Screen
```

The web and mobile applications use the same PHP API and shared MySQL database. The web application can render pages and send AJAX actions. The mobile application sends Axios requests and receives JSON responses. Some mobile data is stored locally before it is synchronized with the server.

Area	Web	Mobile
Page/screen routing	index.php page whitelist	AppNavigator.tsx and role landing logic in MainScreen.tsx
API entry point	ajax.php?action=...	Axios requests to the same ajax.php actions
Business logic	admin_class.php	UI/state preparation plus server-side Action methods
Database	Shared MySQL/MariaDB database	

2. Algorithm 1 — Authentication and Role-Based Routing

Web **Mobile** The system identifies the user, checks the user role, and limits available features. On mobile, the role is stored in AsyncStorage and is used to select the landing screen. On the web, the session role is used by page and API guards and by branch/site filters.

Algorithm

1. Receive username and password.
2. Send credentials to the login operation.
3. Look up the user and verify the credentials.
4. If invalid, return an error and stop.
5. If valid, obtain user ID, role, name, and assigned location.
6. Web: store the values in the PHP session.
7. Mobile: store the values in AsyncStorage and Zustand state.
8. Mobile: route role 8 to POSDashboard, role 9 to PortalScreen, role 10 to OwnerDashboard, and other roles to Dashboard.
9. Web: allow or deny pages/actions according to session role and scope.

Actual implementation: Mobile login is handled by `checkLogin` in `mobile/src/store/globalState.ts`; startup routing is in `mobile/src/screens/MainScreen.tsx`. Web session and role handling are in `payroll/login.php`,

`payroll/index.php`, `payroll/ajax.php`, and `payroll/admin_class.php`.

3. Algorithm 2 — Permission and Local Session Check

Mobile Before opening normal mobile screens, the app checks required Android permissions and whether a saved role exists.

Algorithm

1. Initialize the local database.
2. Check microphone permission.
3. Android 13 or newer: check media image, video, and audio permissions.
4. Older Android: check read and write storage permissions.
5. If a permission is missing, navigate to `PermissionPage`.
6. If permissions are complete and `userType` exists, route by role.
7. If there is no saved `userType`, show the login option.

Actual implementation: `mobile/src/screens/MainScreen.tsx` and `mobile/src/AppNavigator.tsx`.

4. Algorithm 3 — Validation and Normalization

Web **Mobile** Validation prevents incomplete, invalid, or unsafe values from entering the business process. It is applied in the interface and again in important server operations.

Algorithm

1. Receive input from a form, file, local record, or API payload.
2. Check required values.
3. Convert IDs to integers and money/quantity fields to numbers.
4. Check positive dimensions, quantities, and valid date ranges.
5. Check allowed file extensions and parseable date/time values.
6. Reject invalid data with a user-safe message.
7. Normalize valid data into the structure required by the next algorithm.

Actual examples: biometric upload validates `.dat` files and date/time lines in `payroll/admin_class.php`; mobile DTR data is normalized in `mobile/src/store/globalState.ts`; quotation dimensions are checked in `mobile/src/screens/pos/POSQuotationTab.tsx`.

5. Algorithm 4 — Client-Server Request and Response

Web **Mobile** This is the communication algorithm used when either client requests data or submits a change.

Algorithm

1. User performs an action on the web page or mobile screen.
2. Client validates the immediate input.
3. Client sends GET or POST data to `ajax.php?action=operation`.
4. `ajax.php` selects the matching Action method.
5. The Action method validates, reads, or updates MySQL.
6. The server returns HTML/text/JSON according to the operation.
7. Client checks the success/result value.
8. Client shows feedback and refreshes local state or the displayed list.

Actual implementation: Web actions are dispatched in `payroll/ajax.php`. Mobile requests use `mobile/src/utils/axiosConfig.ts` and service/store functions such as `posService.ts` and `globalState.ts`.

6. Algorithm 5 — CRUD Data Processing

Web **Mobile API** The system uses Create, Read, Update, and status-based Delete operations for employees, branches, products, DTR records, quotations, sales, and related records. Newer modules normally deactivate records by changing their status instead of permanently deleting them.

Algorithm

1. Identify the requested entity and operation.
2. Check the user's session or mobile authorization context.
3. Validate and normalize the submitted fields.
4. Read: execute a filtered SELECT query and return the records.
5. Create: insert a new record with the submitted values.
6. Update: identify the record by its ID and update allowed fields.
7. Delete/deactivate: remove the record only where the module permits it; otherwise set its status to inactive.
8. Return a success or error response.
9. Refresh the web table or mobile list.

Actual implementation: Web dispatch is in `payroll/ajax.php`; database methods are in `payroll/admin_class.php`; mobile API calls are made through `mobile/src/utils/posService.ts` and `mobile/src/store/globalState.ts`.

7. Algorithm 6 — Biometric DTR Parsing and Upload

Web server **Mobile sync** The server parses biometric `.dat` files. Mobile DTR records use the same API concept but may be prepared and stored locally before being pushed.

Algorithm

1. Confirm the user is logged in and authorized.
2. Confirm the uploaded file exists and has a .dat extension.
3. Read the file and split it into non-empty lines.
4. Split each line using tab or comma separators.
5. Reject lines with fewer than the required fields.
6. Parse the employee code and date/time.
7. Convert attendance code 0 to In and 1 to Out.
8. Ignore lines with invalid codes or dates.
9. Determine date_from, date_to, branch_id, and device_id.
10. Reject an existing upload with the same date range, branch, and device.
11. Begin a database transaction.
12. Insert the DTR header and each valid DTR detail.
13. Commit if every insert succeeds; roll back on failure.

Actual implementation: payroll/admin_class.php, function upload_biometric_dtr(); mobile preparation and push are in mobile/src/store/globalState.ts, function pushDTR().

8. Algorithm 7 — Payroll Attendance Aggregation and Calculation

Web server **Mobile viewing/reporting** Payroll is calculated on the server from approved DTR details. Mobile displays or reports payroll-related results but does not replace the server payroll calculation.

Algorithm

1. Read payroll period, payroll type, selected branches, and settings.
2. Retrieve approved DTR details inside the payroll dates.
3. Filter records by branch and weekly/monthly payroll type.
4. For each detail, cap work hours at eight hours.
5. Convert hours into payable days.
6. Group details by employee ID.
7. Accumulate work hours, present days, overtime, undertime, and late minutes.
8. Read employee rates, allowances, contributions, deductions, loans, and refunds.
9. Calculate the employee payroll item.
10. Save payroll items in the database transaction.
11. Review and lock the payroll before final reporting.

$$\text{per_hour} = \text{per_day} \div 8$$

$$\text{per_minute} = \text{per_day} \div (8 \times 60)$$

$$\text{regular_pay} = \text{work_hours} \times \text{per_hour}$$

$$\text{overtime_pay} = \text{overtime_hours} \times \text{overtime_rate}$$

$$\text{late_deduction} = \text{late_minutes} \times \text{per_minute}$$

$$\text{net_pay} = \text{gross_pay} - \text{total_deductions} + \text{total_refunds}$$

Actual implementation: `payroll/admin_class.php`, function `calculate_payroll()`; detailed display formulas are also visible in `payroll/payroll_items.php`.

9. Algorithm 8 — POS Sale and Inventory Transaction

Web API **Mobile POS** The mobile POS prepares the cart, but the server recalculates the sale values and saves the sale and stock changes together.

Algorithm

1. Load products for the current branch.
2. Add products and quantities to the mobile cart.
3. Reject an empty cart.
4. On the server, calculate subtotal from price × quantity.
5. Clamp discount between zero and subtotal.
6. Calculate total and change.
7. Begin a database transaction.
8. Insert the sale header and invoice number.
9. Insert all sale item records.
10. Decrease product quantity_on_hand for every item.
11. Read updated stock values.
12. Commit the transaction and return the invoice/result.
13. Roll back if any database operation fails.

$$\text{subtotal} = \Sigma(\text{price} \times \text{quantity})$$

$$\text{total} = \text{subtotal} - \text{discount}$$

$$\text{change} = \max(\text{payment} - \text{total}, 0)$$

Actual implementation: payroll/admin_class.php, function mobile_pos_save_sale(); mobile cart and POS screens are in mobile/src/screens/pos/POSSaleTab.tsx and mobile/src/utils/posService.ts.

10. Algorithm 9 — Quotation Calculation

Mobile calculation

Web API storage

The current quotation screen calculates the displayed quotation in the mobile app. The quotation is then sent to the API for storage. This calculation should be described as a client-side quotation estimate unless the server quotation endpoint is separately changed to recalculate it.

Algorithm

1. Load products and saved quotations.
2. Select a product and enter dimensions and options.
3. Stop with an empty summary when product or dimensions are invalid.
4. Convert the selected unit to square meters and meters.
5. Calculate glass cost using area and design multiplier.
6. Calculate aluminum cost using perimeter.
7. Add selected installation, design, and measurement labor.
8. Apply line-item price overrides.
9. Sum line totals to obtain subtotal.
10. Calculate fixed or percentage discount.
11. Calculate tax on subtotal minus discount.
12. Calculate final total.
13. Add the item to the quotation cart.
14. Send the cart to the quotation API and reload saved quotations.

$$\text{area_m}^2 = \text{width} \times \text{height} \times \text{unit_area_conversion}$$

$$\text{perimeter_m} = 2 \times (\text{width} + \text{height}) \times \text{unit_length_conversion}$$

$$\text{tax} = (\text{subtotal} - \text{discount}) \times \text{tax_percentage} \div 100$$

```
total = subtotal - discount + tax
```

Actual implementation: `mobile/src/screens/pos/POSQuotationTab.tsx` and `mobile/src/utils/quotationCalculator.ts`. API calls are in `mobile/src/utils/posService.ts`.

11. Algorithm 10 — Local Storage and Synchronization

Mobile The mobile application stores selected session, employee, position, department, and DTR data locally. Synchronization transfers selected data between the local database and the central API.

Algorithm

1. Read the saved user and location from AsyncStorage.
2. Request reference data from the API when an online sync is performed.
3. Insert or update downloaded records in the local database.
4. Allow the mobile user to work with local employee or DTR records.
5. Normalize local DTR data before pushing it.
6. Attach the logged-in user and branch/site.
7. Send the payload to the server.
8. If successful, show confirmation and update sync metadata.
9. If unsuccessful, retain local data and show an error.

Actual implementation: `mobile/src/store/globalState.ts`, functions `pullOnline()`, `saveDTR()`, and `pushDTR()`; local database operations are in the mobile utility database service.

12. Algorithm 11 — Report Aggregation

Web server **Mobile owner dashboard** Owner reports use server-side aggregate queries. The mobile dashboard requests the report action and maps the returned count or total to a report card.

Algorithm

1. Owner selects a report.
2. Mobile sends the corresponding owner-report action.
3. Server queries the required count, sum, rate, or branch data.
4. Server returns a JSON result.
5. Mobile checks the success flag.
6. Mobile displays the returned values in the owner dashboard or detail screen.

Actual implementation: mobile report screens in `mobile/src/screens/OwnerDashboard.tsx` and `POSOwnerReportsTab.tsx`; report actions are dispatched in `payroll/ajax.php`.

13. Accuracy and Boundary Notes

- Payroll calculation is performed by the PHP server from approved DTR records.
- POS sale totals are recalculated server-side before saving.
- Biometric parsing and DTR upload validation are performed by the PHP server.
- Quotation preview calculation is performed in the current mobile quotation screen before API storage.
- Mobile role routing is performed by the React Native app after login or saved-session detection.

- Web access uses PHP session and role checks; mobile access uses the mobile login response and stored role/location values.
- Transactions are used for multi-step operations so related database changes commit or roll back together.

Prepared from the current implementation in `c:\xampp\htdocs\payroll` and `c:\xampp\htdocs\mobile`. Update this guide whenever an algorithm or API responsibility changes.